Table of Contents

1 Introduction....1
2 Top-down layered static view....2
3 Detailed design of the gpio_service module....3
3.1 LEDControl Class....3
3.1.1 Purpose....3
3.1.2 Design & Threading....3
3.1.3 API Reference....4
3.1.3.1 LEDS Constants....4
3.1.3.2 Methods:....4
3.1.4 Error handling....6
3.2 Module test....6

1 Introduction

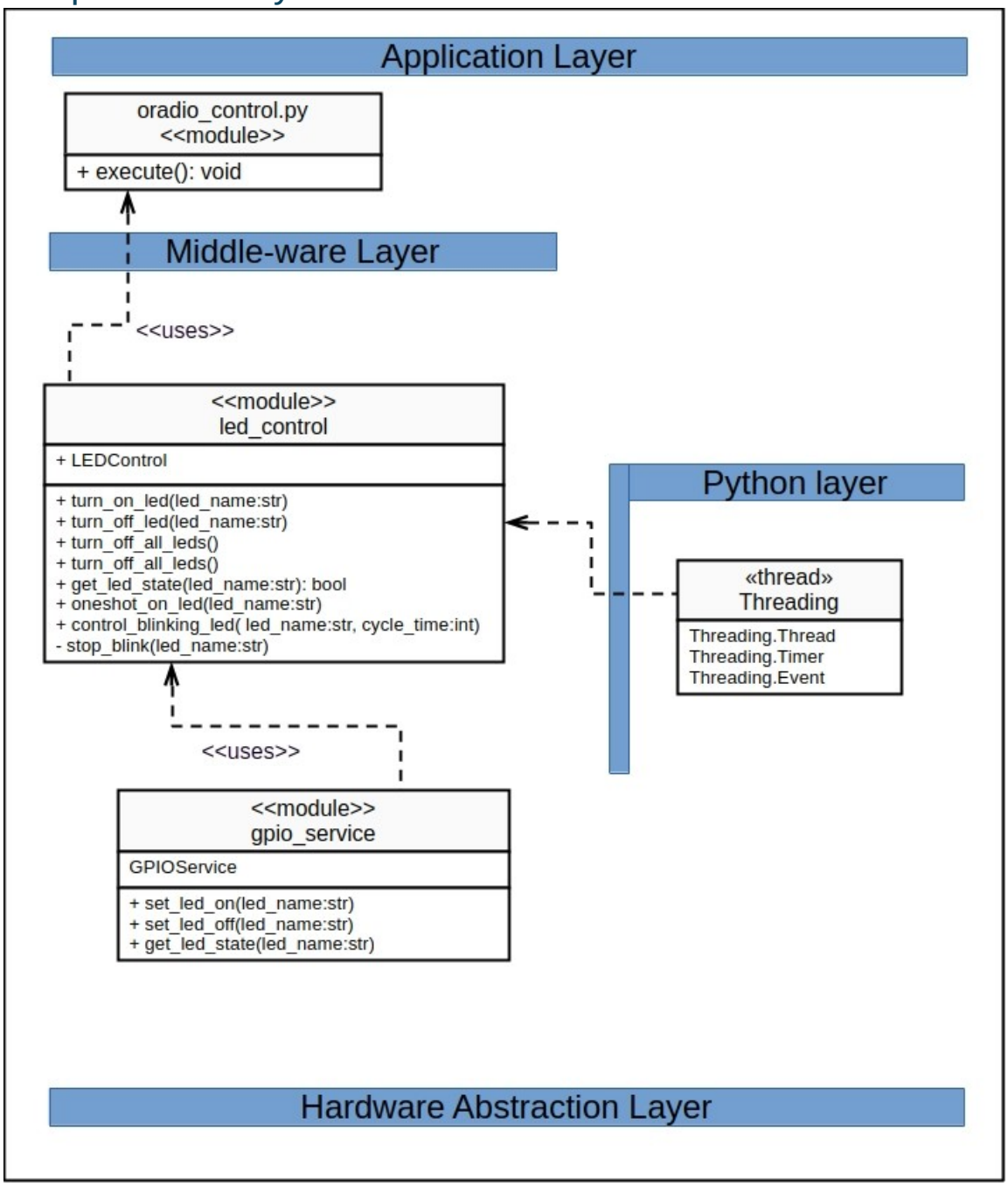
The LED's on the front panel are behind the touch buttons. The main functions of the LED's are:

- During power-ON-boot cycle the STOP_LED is on, to indicate that boot cycle is active.
- When Oradio is being initialized during starting up, the STOP_LED is blinking until Oradio is ready to use.
- All LED's are off (or in backlight off) when Oradio is ready.
- When a touch button is touched or pressed the related touch button led will be on.
- To indicate a special state one of the LED's may start blinking:
  - When Oradio Access Point is active: the PLAY button is blinking
  - When an error condition is found: the STOP button is blinking

There is a background process (backlight) that manages the "off" and "on" light level of the LED's. This backlight level is controlled by a daylight sensor, to improve the visibility of the LED's when environmental light conditions are changing.

The backlight control is not part of this description.

2 Top-down layered static view



The Oradio3 software layering typically consists of three main layers:

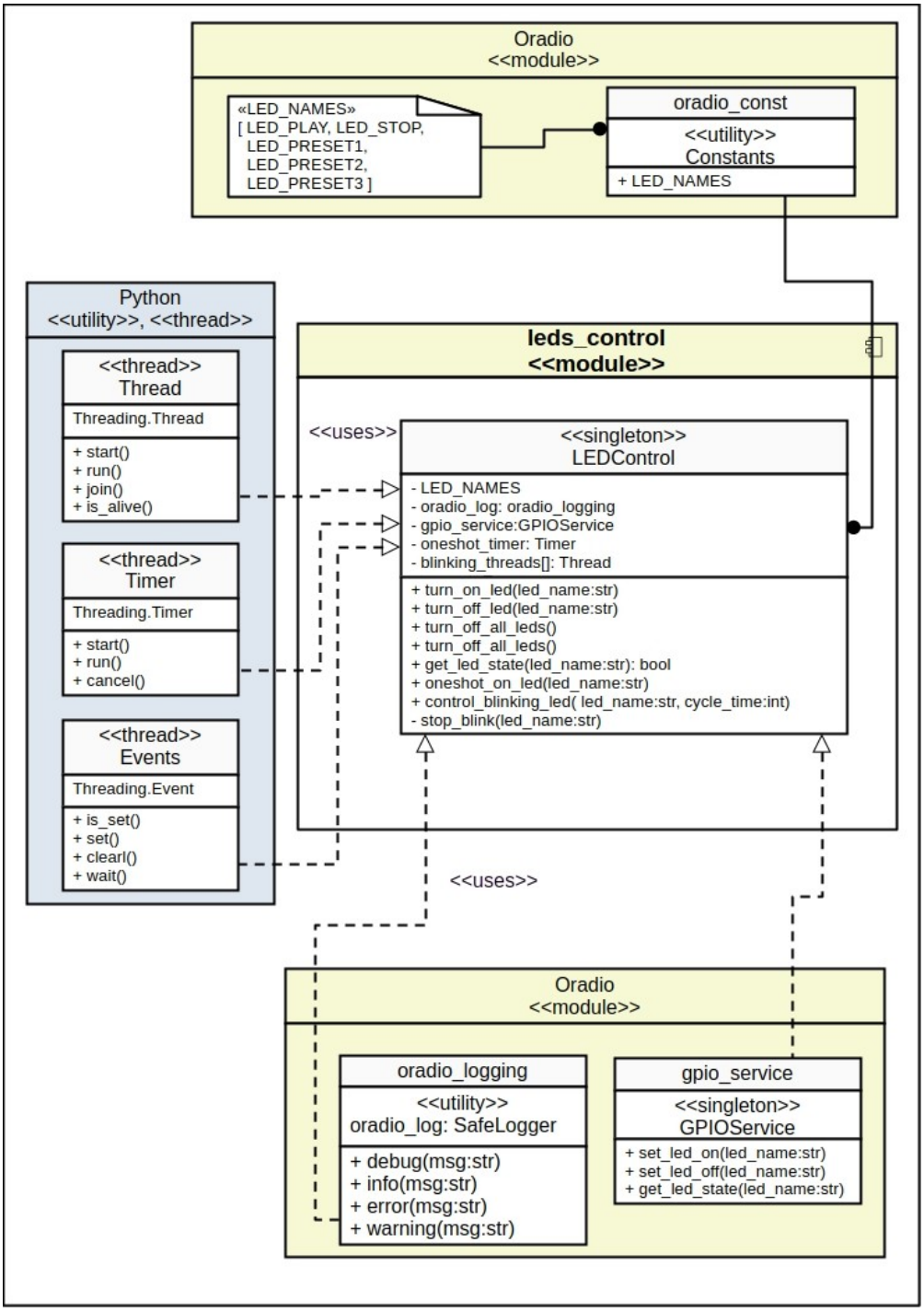
- the **hardware abstraction layer** (HAL), which includes the physical components.
- the **middle-ware layer**, which facilitates communication and data management between the hardware and applications. The OS services is considered to be part of this layer
- the **application layer**, where the actual software applications run and perform specific tasks.

The **Python Layer** is a supporting layer to include Python modules and libraries.

The LEDControl class is a Oradio3 module offering services to turn on or off a specific LED. It imports the GPIOService class, which provides a wrapper for the physical writing to I/O pins assigned to the LED's

The LEDControl uses the Python Threading for timers, threads and event handling.

3 Detailed design of the led_control module.



3.1 LEDControl Class

3.1.1 Purpose

- Provides methods to control the state of the LED's
- Manages and reports (logging) LED's related errors and warnings.

3.1.2 Design & Threading

- **Singleton:** Restricts the class to a single instance, to offer a single point of access for the led controls.

- **Threads:** Blinking support for each LED is managed by a thread. Each LED blinking thread is running until it is stopped. The blinking thread uses an eventing mechanism to turn on and off the led periodically.
- **Timers**: For the turning on a led for a specific period (one-shot) a Threading.timer is used
- **Events**: The blinking thread uses Event.wait() function for a half-period to turn a led on or off. As long as the event is active the eventing proceeds in the thread. When event is stopped the thread is also stopped.
- **Logging**: For logging purposes the oradio.SafeLogger utility is used offered by oradio_logging module.
- **GPIOService**: creates an instance of the GPIOService class to use methods which sets the physical LED-pin on or off.

Design decision:

The blinking of a led is started with **method control_blinking_led().** The blinking is stopped by the application by either turning on or off the related led. There is no method provided to stop the blinking.

3.1.3 API Reference

3.1.3.1 LEDS Constants

```
LED_PLAY    = "LedPlay"
LED_STOP    = "LedStop"
LED_PRESET1 = "LedPreset1"
LED_PRESET2 = "LedPreset2"
LED_PRESET3 = "LedPreset3"
LED_NAMES   =[LED_PLAY, LED_STOP, LED_PRESET1, LED_PRESET2, LED_PRESET3]
```

3.1.3.2 Methods:

Method	Arguments	Returns
`LEDControl.turn_off_led`	`(led_name: str)`	None
Description: • Turning OFF of the specified led (led_name). • led_name = `[LED_PLAY | LED_STOP | LED_PRESET1 | LED_PRESET2 | LED_PRESET3]`		
Errors: • *`("Invalid LED name: %s", led_name)`*		

Method	Arguments	Returns
`LEDControl.turn_on_led`	`(led_name: str)`	None
Description: • Turning ON of the specified led (led_name). • led_name = `[LED_PLAY | LED_STOP | LED_PRESET1 | LED_PRESET2 | LED_PRESET3]`		
Errors:		

• *("Invalid LED name: %s", led_name)*

Method	Arguments	Returns
LEDControl.turn_off_all_leds	()	None
Description: Turning OFF all led's		
Errors:		

74

Method	Arguments	Returns
LEDControl.turn_on_all_leds	()	None
Description: Turning ON all led's		
Errors:		

76

Method	Arguments	Returns
LEDControl.oneshot_on_led	(led_name: str, period: float=3)	None
Description: Turns ON specified led (led_name) for specified duration (period) and the turns off the specified led. • led_name = [LED_PLAY | LED_STOP | LED_PRESET1 | LED_PRESET2 | LED_PRESET3] • period = a float, with as default value 3		
Errors: • *("Invalid LED name: %s", led_name)* • *("Invalid period time of %f for one-shot of led: %s", period, led_name)*		

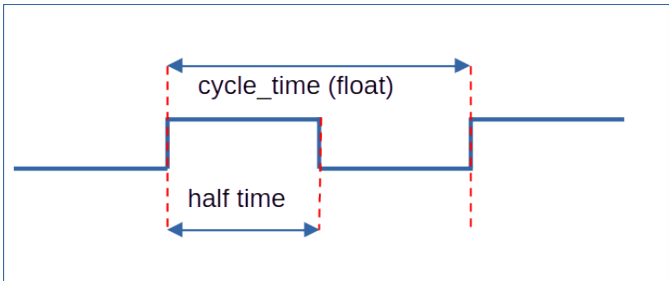
78

Method	Arguments	Returns
LEDControl.get_led_state	(led_name: str)	• True for led in ON state, • False for led in OFF state • None for unknown led name
Description: Reads and returns the actual state of a led • led_name = LED_PLAY | LED_STOP | LED_PRESET1 | LED_PRESET2 | LED_PRESET3]		
Errors: • *("Unknown LED name: %s", led_name)*		

80

Method	Arguments	Returns
LEDControl.control_blinking_led	(led_name: str, cycle_time: float=2)	None

Description:
Start blinking of specified led (led_name)



- led_name = [LED_PLAY | LED_STOP | LED_PRESET1 | LED_PRESET2 | LED_PRESET3]
- cycle_time = a float, with as default value 2

Errors:

- `("Invalid LED name: %s", led_name)`

3.1.4 Error handling

- All errors are logged using the oradio_log.error or .warning utility of oradio_logging.py

3.2 Module test

There is a module test program <led_control_test.py> that will provide following tests to test the methods of the class:

- All led's ON or OFF
- All led's blinking, with adjustable cycle-time
- A one-shot cycle for All led's, with adjustable one-shot timing
- A single led test:
  - Turn ON or OFF a selected led,
  - One-shot for a selected led,
  - Blinking for a selected led.
  - Unknown led error handling